



SOLID Principles

for Absolute Beginners — a friendly, no-jargon guide with JavaScript examples

ZERO PRIOR KNOWLEDGE REQUIRED

Written for developers coming from Express.js / Node.js

Contents

0

What Even Is SOLID?

The big picture before we dive in

S

Single Responsibility Principle

One class, one job

O

Open/Closed Principle

Add new features without editing old code

L

Liskov Substitution Principle

Subclasses must keep their parent's promises

I

Interface Segregation Principle

Don't force unused behavior on a class

D

Dependency Inversion Principle

Depend on ideas, not specific tools

6

Putting It All Together

All five principles in one worked example

7

Common Beginner Questions

FAQs and over-engineering warnings

8

One-Page Cheat Sheet

Everything at a glance

Chapter 0 • What Even Is SOLID?

SOLID is not a tool, a library, or a framework. It's just **5 ideas** about how to organize your code so that:

- It's easy to change without breaking other things
- It's easy for someone else (or future-you) to understand
- It's easy to test
- It doesn't turn into a giant tangled mess as it grows

Letter	Principle	One-line idea
S	Single Responsibility Principle	One class/function should do one job
O	Open/Closed Principle	Add new features without editing old code
L	Liskov Substitution Principle	A subtype should be swappable for its parent without breaking things
I	Interface Segregation Principle	Don't force code to depend on things it doesn't use
D	Dependency Inversion Principle	Depend on general ideas , not specific implementations

Don't worry if that table means nothing right now — that's what the rest of the book is for. By the end, you'll re-read this table and it will actually make sense.

Important beginner note: SOLID was designed for **object-oriented programming** (code built around classes and objects). Since you already write Express.js, you've used objects and functions constantly — you just haven't organized them with these specific rules yet. All examples in this book use plain JavaScript classes, which work the same in Node.js as anywhere else.

Every principle in the chapters ahead follows the same pattern: a plain-English explanation, a real-world analogy, bad code, why it's bad, good code, and a quick recap. Take your time — don't rush from one principle to the next.

S

Single Responsibility Principle

Chapter 1

"A class (or function, or file) should have only one reason to change."

PLAIN ENGLISH

Give each piece of code **one job**. Not two, not "one main job and a couple of side jobs." One.

REAL-WORLD ANALOGY

Restaurant: Imagine a restaurant where the same person is the chef, the waiter, the cashier, AND the cleaner. If the menu changes, that person is affected. If the payment system changes, that person is affected. If health & safety rules change, that person is affected. A well-run restaurant splits these into separate roles — a chef, a waiter, a cashier — so a change in one role doesn't ripple into the others.

BAD CODE

✗ BAD

```
// user.js
class User {
  constructor(name, email) {
    this.name = name;
    this.email = email;
  }

  // Job 2: saving to database
  saveToDatabase() {
    console.log(`INSERT INTO users (name, email) VALUES ('${this.name}', '${this.email}')`);
  }

  // Job 3: sending emails
  sendWelcomeEmail() {
    console.log(`Sending welcome email to ${this.email}`);
  }

  // Job 4: validating data
  validateEmail() {
    return this.email.includes('@');
  }
}
```

WHY THIS IS BAD

This `User` class has **4 reasons to change**: switching databases, switching email providers, changing validation rules, or changing what user data looks like. Six months from now, a teammate fixing a bug in `saveToDatabase()` could accidentally break `sendWelcomeEmail()` because everything shares the same class. Bugs "leak" across unrelated responsibilities, and testing becomes painful — testing validation alone drags in database and email code too.

GOOD CODE

✓ GOOD

```
// user.js — ONLY represents user data
class User {
  constructor(name, email) {
    this.name = name;
    this.email = email;
  }
}

// userRepository.js — ONLY handles saving/loading users
class UserRepository {
  save(user) {
    console.log(`INSERT INTO users (name, email) VALUES ('${user.name}', '${user.email}')`);
  }
}

// emailService.js — ONLY handles sending emails
class EmailService {
  sendWelcomeEmail(user) {
    console.log(`Sending welcome email to ${user.email}`);
  }
}

// userValidator.js — ONLY handles validation rules
class UserValidator {
  isValidEmail(user) {
    return user.email.includes('@');
  }
}
```

```
const user = new User('Subhajit', 'subhajit@example.com');
const validator = new UserValidator();
const repository = new UserRepository();
const emailService = new EmailService();

if (validator.isValidEmail(user)) {
  repository.save(user);
}
```

```
emailService.sendWelcomeEmail(user);  
}
```

Why it's better: change your database logic and you only touch `UserRepository`. Switch email providers and you only touch `EmailService`. Each class is small, readable in ten seconds, and easy to test alone.

BEGINNER TRAP

SRP does **not** mean "every class should have only one method." It means "every class should have only one **reason to change**." A `UserRepository` can have `save()`, `findById()`, `delete()` — all still the *same* responsibility (managing user storage), so they belong together.

QUICK RECAP

If you can't describe what a class does without using the word "and," it's probably doing too much.



Open/Closed Principle

Chapter 2

"Software entities should be open for extension, but closed for modification."

PLAIN ENGLISH

You should be able to **add new behavior** without **editing existing, tested code**. You extend it, not rewrite it.

REAL-WORLD ANALOGY

Power strip: If you want to plug in a new device, you don't rewire the wall socket — you just plug into an open slot. The socket is "closed" (you don't touch it), but the system is "open" for extension (you can add more devices).

BAD CODE

✗ BAD

```
class DiscountCalculator {
  calculate(customerType, price) {
    if (customerType === 'regular') {
      return price;
    } else if (customerType === 'member') {
      return price * 0.9;
    } else if (customerType === 'vip') {
      return price * 0.8;
    }
    // What happens when marketing adds a new "student" discount?
  }
}
```

WHY THIS IS BAD

Every time your business adds a new customer type, you have to **open this file and edit it**. You might break the existing chain by mistake, every edit needs a re-test of the entire function, and this function keeps growing until it's an unreadable wall of `else if` blocks.

GOOD CODE

✓ GOOD

```
// Each discount type is its own small class
class RegularDiscount {
  apply(price) { return price; }
}
class MemberDiscount {
  apply(price) { return price * 0.9; }
}
class VipDiscount {
  apply(price) { return price * 0.8; }
}

// The calculator doesn't know or care about specific discount types
class DiscountCalculator {
  calculate(discountStrategy, price) {
    return discountStrategy.apply(price);
  }
}
```

```
const calculator = new DiscountCalculator();
console.log(calculator.calculate(new MemberDiscount(), 1000)); // 900
console.log(calculator.calculate(new VipDiscount(), 1000));    // 800

// Adding "student" discount tomorrow — DiscountCalculator is NEVER touched:
class StudentDiscount {
  apply(price) { return price * 0.85; }
}
console.log(calculator.calculate(new StudentDiscount(), 1000)); // 850
```

Why it's better: DiscountCalculator never changes again, no matter how many discount types you add. You only ever add new files/classes — far safer than editing old, working, tested code.

BEGINNER TRAP

You don't need to apply OCP to *everything* on day one — that leads to over-engineering. Apply it when you notice a pattern of "I keep adding new if/else branches here every few weeks." That's the signal.

QUICK RECAP

If adding a new feature means editing an old file's core logic, look for a way to add a new file instead.



Liskov Substitution Principle

Chapter 3

"Objects of a subclass should be replaceable with objects of the parent class, without breaking the program."

This one sounds scary, but it's simple once you see it.

PLAIN ENGLISH

If class B extends class A, then anywhere your code uses A, you should be able to swap in B instead — and everything should still work correctly, with no surprises.

REAL-WORLD ANALOGY

Toy car: If someone tells you "any vehicle can be driven," and then you're handed a toy car that doesn't actually move — that toy breaks the promise a "vehicle" is supposed to keep. It calls itself a vehicle, but can't do what vehicles are expected to do. That's an LSP violation.

BAD CODE

✗ BAD

```
class Bird {
  fly() { console.log('Flying high!'); }
}

class Sparrow extends Bird {
  fly() { console.log('Sparrow flying'); }
}

class Penguin extends Bird {
  fly() { throw new Error("Penguins can't fly!"); } // uh oh
}

function makeBirdFly(bird) {
  bird.fly();
}

const birds = [new Sparrow(), new Penguin()];
birds.forEach(makeBirdFly); // ✖ crashes on Penguin
```

WHY THIS IS BAD

Penguin is technically a `Bird`, but it **breaks the contract** that "every `Bird` can `fly()`." Any code trusting `Bird.fly()` will randomly crash the moment a Penguin shows up. The bug isn't obvious from reading `makeBirdFly()` — it only appears at runtime, which makes it sneaky and hard to debug.

GOOD CODE

✓ GOOD

```
class Bird {
  eat() { console.log('Eating...'); }
}

class FlyingBird extends Bird {
  fly() { console.log('Flying high!'); }
}

class Sparrow extends FlyingBird {
  fly() { console.log('Sparrow flying'); }
}

class Penguin extends Bird {
  swim() { console.log('Penguin swimming'); }
}

function makeBirdFly(bird) {
  bird.fly();
}

const flyingBirds = [new Sparrow()];
flyingBirds.forEach(makeBirdFly); // ✓ safe — only FlyingBirds are passed here
```

Why it's better: Penguin never promises to fly. Code expecting flying behavior only ever receives things that can actually fly — no surprise crashes, no need to remember special cases.

BEGINNER TRAP

LSP violations often *look* fine at first — the code runs, no errors shown. The problem shows up later, in production, when someone passes in the "wrong" subclass. Watch for: a subclass method that throws an error, does nothing, or needs a special `if` check elsewhere to avoid calling it.

QUICK RECAP

If you have to ask "but what if it's actually a Penguin?" somewhere in your code,

your class hierarchy has an LSP problem.



Interface Segregation Principle

Chapter 4

"Don't force code to depend on methods/behaviors it doesn't need."

Note for JS devs: JavaScript doesn't have formal "interfaces" like Java or C#. Think of an "interface" here as "the set of methods something expects an object to have."

PLAIN ENGLISH

Don't design one giant do-everything contract. Break it into smaller, focused ones, so a class only has to implement what it actually uses.

REAL-WORLD ANALOGY

Job application form: Imagine a form that asks EVERY applicant — chef, driver, or accountant — to fill in "years of driving experience," "cooking certifications," AND "accounting license number." A driver applicant awkwardly leaves 2/3 of the form blank. A good process shows each applicant only the fields relevant to their role.

BAD CODE

✗ BAD

```
class Worker {
  work() { throw new Error('Not implemented'); }
  eat() { throw new Error('Not implemented'); }
  attendMeeting() { throw new Error('Not implemented'); }
}

class HumanEmployee extends Worker {
  work() { console.log('Working...'); }
  eat() { console.log('Eating lunch...'); }
  attendMeeting() { console.log('In a meeting...'); }
}

class RobotWorker extends Worker {
  work() { console.log('Working...'); }
  eat() {
    // Robots don't eat! But we're FORCED to implement this
    throw new Error('Robots do not eat');
  }
  attendMeeting() {
    throw new Error('Robots do not attend meetings');
  }
}
```

```
}
```

WHY THIS IS BAD

RobotWorker is forced to "implement" `eat()` and `attendMeeting()` even though those make no sense for a robot. Any code looping over a list of workers and calling `.eat()` on all of them will crash the moment it hits a robot — unless you scatter `if` checks everywhere, which defeats the purpose of a shared class.

GOOD CODE

✓ GOOD

```
// Split into small, focused capabilities
class Workable {
  work() { throw new Error('Not implemented'); }
}
class Eatable {
  eat() { throw new Error('Not implemented'); }
}
class MeetingAttendee {
  attendMeeting() { throw new Error('Not implemented'); }
}

// HumanEmployee uses all three — humans need all of them
class HumanEmployee extends Workable {
  work() { console.log('Working...'); }
  eat() { console.log('Eating lunch...'); }
  attendMeeting() { console.log('In a meeting...'); }
}

// RobotWorker only implements what actually applies to it
class RobotWorker extends Workable {
  work() { console.log('Working 24/7...'); }
}
```

Why it's better: RobotWorker is no longer forced to fake-implement methods that don't apply. Code that only cares about "things that can work" depends on `Workable` alone, without worrying about eating or meetings.

BEGINNER TRAP

ISP is easy to confuse with SRP. SRP is about a class doing too many jobs *internally*. ISP is about a class being *forced* to support methods it doesn't need, because of a bloated shared contract.

QUICK RECAP

If a class has methods that just throw errors or do nothing on purpose, the shared contract is too fat — split it up.



Dependency Inversion Principle

Chapter 5

"Depend on abstractions (general ideas), not on concrete (specific) implementations."

This is the one that trips up beginners the most — but it's also the most useful in real projects like yours (Express.js APIs, database access, etc).

PLAIN ENGLISH

Instead of your code being tightly wired to one specific tool (e.g. "MySQL specifically"), wire it to a general idea (e.g. "a database, whichever one it is") — and plug in the specific tool from outside.

REAL-WORLD ANALOGY

Lamp: A lamp doesn't care whether electricity comes from a coal plant, a solar panel, or a wind turbine. It just needs "electricity through a standard socket." The power source can change completely without redesigning the lamp.

BAD CODE

✗ BAD

```
class MySQLDatabase {
  save(data) {
    console.log(`Saving to MySQL: ${JSON.stringify(data)}`);
  }
}

class OrderService {
  constructor() {
    this.database = new MySQLDatabase(); // ⚬ hardcoded, specific dependency
  }

  placeOrder(order) {
    this.database.save(order);
  }
}
```

WHY THIS IS BAD

OrderService is welded to MySQLDatabase. Switching to MongoDB later means editing

OrderService directly. Testing `placeOrder()` without a real database becomes impossible. And OrderService — which should only care about order business logic — now knows database details it shouldn't.

GOOD CODE

✓ GOOD

```
// The "abstraction" — a general shape any database must follow
class Database {
  save(data) { throw new Error('save() must be implemented'); }
}

class MySQLDatabase extends Database {
  save(data) { console.log(`Saving to MySQL: ${JSON.stringify(data)}`); }
}

class MongoDBDatabase extends Database {
  save(data) { console.log(`Saving to MongoDB: ${JSON.stringify(data)}`); }
}

// OrderService depends on the GENERAL idea "Database", not a specific one
class OrderService {
  constructor(database) {
    this.database = database; // ✓ handed in from outside ("injected")
  }
  placeOrder(order) {
    this.database.save(order);
  }
}
```

```
const mysqlService = new OrderService(new MySQLDatabase());
mysqlService.placeOrder({ item: 'Laptop', price: 50000 });

const mongoService = new OrderService(new MongoDBDatabase());
mongoService.placeOrder({ item: 'Mouse', price: 500 });
```

Why it's better: switching databases doesn't require touching OrderService at all — just hand it a different object. Testing becomes trivial: hand it a fake database that just records what was "saved." This pattern is called **Dependency Injection** — it's literally how you achieve Dependency Inversion in practice, and you'll see it all over real Node.js/Express backends.

BEGINNER TRAP

"Dependency Inversion" does NOT mean "avoid dependencies." You still depend on things — you just depend on a general shape/contract instead of one hardcoded concrete class, and the specific choice gets "injected" from outside.

QUICK RECAP

If your class creates its own dependencies with `new SomeSpecificThing()` internally, ask: "could I hand this in from outside instead?"

6

Putting It All Together

Chapter 6

Here's a tiny before/after that touches all 5 principles, using something close to your world — a simple order processing flow.

BEFORE — VIOLATES ALMOST EVERYTHING

X BAD

```
class OrderManager {
  constructor() {
    this.db = new MySQLDatabase(); // DIP violation: hardcoded dependency
  }
  processOrder(order) {
    // validation, discount calculation, saving, AND emailing —
    // all crammed into one method (SRP violation)
    if (!order.email.includes('@')) throw new Error('Invalid email');

    if (order.customerType === 'vip') order.price *= 0.8;
    else if (order.customerType === 'member') order.price *= 0.9;
    // adding a new customer type means editing this if/else (OCP violation)

    this.db.save(order);
    console.log(`Emailing confirmation to ${order.email}`);
  }
}
```

AFTER — SOLID APPLIED

✓ GOOD

```
// --- Single Responsibility: one job per class ---
class OrderValidator {
  isValid(order) { return order.email.includes('@'); }
}
class EmailService {
  sendConfirmation(order) { console.log(`Emailing confirmation to ${order.email}`); }
}

// --- Open/Closed: new discount types = new classes, not new if/else ---
class VipDiscount { apply(price) { return price * 0.8; } }
class MemberDiscount { apply(price) { return price * 0.9; } }
class NoDiscount { apply(price) { return price; } }

// --- Dependency Inversion: general "Database" shape, injected in ---
class Database { save(data) { throw new Error('Not implemented'); } }
class MySQLDatabase extends Database {
  save(data) { console.log(`Saving to MySQL: ${JSON.stringify(data)}`); }
}
```

```
// --- OrderManager now only ORCHESTRATES ---
class OrderManager {
  constructor(database, validator, emailService) {
    this.database = database;
    this.validator = validator;
    this.emailService = emailService;
  }
  processOrder(order, discountStrategy) {
    if (!this.validator.isValid(order)) throw new Error('Invalid email');
    order.price = discountStrategy.apply(order.price);
    this.database.save(order);
    this.emailService.sendConfirmation(order);
  }
}

const manager = new OrderManager(
  new MySQLDatabase(),
  new OrderValidator(),
  new EmailService()
);

manager.processOrder(
  { email: 'subhajit@example.com', price: 1000 },
  new VipDiscount()
);
```

Notice: every class here is short enough to understand in a few seconds, and every future change (new discount, new database, new validation rule) means **adding** something, not **rewriting** something that already works.

Q: Do I need to use all 5 principles in every single project, even small ones?

No. For a quick script or tiny personal project, applying all of SOLID strictly is overkill. SOLID earns its value as a project **grows** — more files, more developers, more features over time. Learn to recognize the smells (long if/else chains, giant classes, hardcoded `new SomeClass()` everywhere) and apply the relevant principle when you actually feel the pain.

Q: Is SOLID only for object-oriented languages?

It was designed with OOP in mind, but the underlying ideas — single responsibility, avoiding hardcoded rewrites, small focused contracts — show up in functional programming too, just with different vocabulary.

Q: I write Express.js route handlers. Where does SOLID fit in?

Very naturally. A common pattern: your route handler (controller) calls a "service" (business logic), which calls a "repository" (database access) — that's SRP in action. Passing a database connection into a service instead of creating it inside the service — that's DIP.

Q: How do I know if I'm "over-engineering" with SOLID?

If you're creating abstractions for something that will realistically never change or never need a second implementation, you're probably over-engineering. A good rule of thumb: apply OCP/DIP when you've had to change the same piece of logic 2+ times already, not preemptively for hypothetical futures.

Principle	Ask yourself	Red flag
SRP	"Can I describe this class's job without saying 'and'?"	Class does validation + saving + emailing, all in one
OCP	"Do I need to edit old code to add a new feature?"	Long if/else or switch that grows every few weeks
LSP	"Can I swap the child class in without surprises?"	Subclass method throws an error or does nothing
ISP	"Does this class implement methods it doesn't need?"	Empty or error-throwing method implementations
DIP	"Does my class create its own dependencies with new X()?"	Hardcoded new MySQLDatabase() instead of injecting it

What to Do Next

1. Open one of your existing Express.js projects.
2. Pick your biggest, messiest file (usually a controller or a "utils.js").
3. Ask: "What are all the different *reasons* this file might need to change?" — that's your SRP audit.
4. Split just **one** responsibility out into its own file/class. That's it. Don't refactor everything at once.

SOLID isn't something you memorize and apply perfectly overnight — it's a lens you slowly get used to looking through. Come back to this book whenever a class starts feeling "too big" or "too tangled," and match the smell to the right chapter.